Systemudvikling og softwa-rearkitektur

Marts 2026

Fornavn Efternavn **Fornavn Efternavn** **Fornavn Efternavn**

Teknisk Rapport
Datamatiker uddannelsen
25. March 2026

Formalia
Antal anslag: 8448
Antal normalsider: 3.5

Indhold

1 Opstart og Projektstrategi . . . 1
1.1 Det kunstige problemdomæne . . . 1
1.2 Opgaveformulering vs. agil metode . . . 1
1.3 Iterationsstruktur . . . 2
1.4 Slices . . . 2
2 Planlægning og tidsestimering . . . 3
2.1 Value- og risikodrevet . . . 3
2.2 Konsekvenser for Task-formuleringer . . . 3
3 Forløbet . . . 4
3.1 Faktiske iterationsstruktur . . . 4
4 Arkitektur og design . . . 6
4.1 Composite Pattern . . . 6
4.2 Facade Pattern . . . 9
5 Diskussion . . . 10
5.1 Andre Patterns . . . 10
5.2 SOLID og GRASP . . . 10
6 Refleksion . . . 11
6.1 Agil metode i et læringsscenarie . . . 11
6.2 Task-formuleringer . . . 11
7 Litteraturliste . . . 12
8 Bilag . . . 13

1 Opstart og Projektstrategi

1.1 Det kunstige problemdomæne

Vores projekt har fokuseret på en case bestående af *Bostedet Tulipanvej* der huser mentalt udfordrede borgere. Derudover tilbyder stedet bostøtte, hvilket er en støtteordning som borgerne modtager i hjemmet, som en overgang til at kunne bo selvstændigt i egen bolig. Vores domænekendskab bygger på en privat samtale vi havde med en specialpædagog, Laura, der arbejder i bostøtten. Ud fra den lærte vi at pædagogerne har ansvaret for at koordinere og planlægge bogerbesøgene og hvem der bruger deres begrænsede antal biler hvornår. Denne ad hoc koordinering er kritisk for arbejdsgangen, da de supportere borgere i hele Silkeborg kommune. Senere modtog vi en SMS fra Laura (Afsnit Bilag A -), hvorfra vi kunne udlede:

- 14 pædagoger
- 4 cykler delt med botilbuddet
- 4 biler kun til bostøtten
- Booking (af biler) sker i Outlook
- Der er retningslinjer for, hvor langt frem man må booke

På trods af denne information måtte vi erkende at vores kendskab til domænet var for begrænset til at vi kunne gå igang med en Use Case Workshop som Inception fasen oftest startes med i Unified Process (Larman, u.å.). Vores ønske var at arbejde med et ægte domæne for at kunne arbejde agilt, men vi blev nød til at tilgå det som et kunstigt domæne. Derfor valgte vi at ændre på processen og starte med at modellere problemdomænet med en koncept model. Det gjorde vi for at få et fælles grundlag for det (delvist) fiktive domæne. Normalt ville modelleringen komme efter at domænet er blevet afdækket med en PO i en use case workshop. I vores projekt brugte vi det som et værktøj til at blive enige om hvlilke antagelser vi ville lave om de manglende domæne information.

1.2 Opgaveformulering vs. agil metode

Det kunstige scenarie tvang os til at reflektere over, hvordan man arbejder agilt i en kontekst med stærkt begrænsede ressourcer og tid. Dilemmaet bestod i, om vi skulle lade projektets rammer — et to-ugers skoleprojekt — begrænse vores analyse fra start, eller om

vi skulle tilgå processen med et åbent sind og analysere domænet naivt, som om det var et fuldtskala-projekt.

Vi valgte at forblive tro mod den agile tankegang og gennemføre analysen uden forudindtagede begrænsninger. For at håndtere det begrænsede scope i praksis, inddrog vi det som en faktor i vores prioritering frem for i analysen. Konkret betød det, at use cases der lå uden for realistisk rækkevidde, blev tildelt en høj værdi på *Cost*-aksen i vores *value-driven* prioritering. Derved sikrede vi, at prioriterings-scoren naturligt sorterede urealistiske krav fra, samtidig med at vi opretholdt retten til pragmatisk at ekskludere use cases, der trods en favorabel score ikke kunne rummes inden for tidsrammen.

1.3 Iterationsstruktur

Vi tænker, at inception-fasen skal løbe over to dage (torsdag 12/3 + fredag 13/3)

Vi arbejder ud fra klients første domænebeskrivelse.

Vi diskuterer, om 1-3 iterationer er realistisk i projektet.

Inception oversigt

^1c3ba6

- OOA
- Domænemodel
- Krav-workshop - Use Case Model
- prioriter krav - ud fra risici og kundeværdi
- planlæg iterationer (Vi lægger en overordnet plan - men holder 'mandagsmøder' og 'opsamlingsmøder' i starten og slutningen af hver iteration med kommunikations-UML'er)

Vi diskuterer den agile proces i så kort et projekt. Vi tænker, at der skal være et par hele dagen til at lave færdige diagrammer og renskrive opgaven.

WPF er forholdsvist nyt for gruppen - det taget i betragtning tænker vi 3 iterationer på to dage hver er udgangspunktet

Dag 1 Elaboration starter mandag (16/3)

Vi diskuterer burnout-chart vs Gantt kort

1.4 Slices

2 Planlægning og tidsestimering

2.1 Value- og risikodrevet

2.2 Konsekvenser for Task-formuleringer

3 Forløbet

3.1 Faktiske iterationsstruktur

Den oprindelig plan bestod af tre iterationer, hvor hver iteration tog udgangspunkt i én use case opdelt i konkrete tasks. Det viste sig hurtigt, at vores første planlagte iteration (UC2: Se ledige transportmidler), indeholdt væsentligt mere kompleksitet end forventet, og ikke kunne gennemføres med en to-dages iteration.

Allerede på første dag af iteration 1 (mandag d. 16 marts) stod det klart, at inception-fasen ikke var tilstrækkeligt afsluttet. Derfor prioriterede vi at afsætte tid til at færdiggøre estimering og task-formulering frem for at påbegynde implementering på et uklart grundlag.

Vores opdeling i feature-orienterede tasks samt beslutningen om at arbejde med vertikale slices (Larman, u.å.) medførte mange arkitektoniske beslutninger blev truffet tidligt i forløbet.

Iteration 1 - UC 2 (16.-17. marts) (24 story points): Tasks:

De definerede tasks varierede både i kompleksitet og detaljegrad. Selvom de mest abstrakte tasks (f.eks. “Opret Bookingfunktion”) var frasorteret på dette tidspunkt, viste det sig at de resterende tasks dække over større tekniske beslutninger end forventet.

Vi prioriterede derfor de mest håndgribelige tasks: “Brugerdata” og “Håndter data for transportmidler”. Disse tasks, som oprindeligt var estimeret til 2 story points hver, kom reelt til at udgøre hovedparten af iterationen.

Implementeringen af bruger- og køretøjsdata dækkede reelt over et helt lag af beslutninger hveriblandt filformat, serialisering, repository-pattern og interfacedesign. Som følge heraf blev selve implementeringen implementeringen først påbegyndt d. 17. marts efter vi havde fastalgt en lagdelt arkitektur bestående af Domæne-, IO- og Services-lag.

Den efterfølgende task, “Se ledige køretøjer”, blev påbegyndt med henblik på at indrage præsentationslaget. Tasken var estimeret til 8 story points, men blev kun delvist implementeret inden for iterationen. Vi besluttede derfor at lade denne task fortsætte ind i næste iteration.

Iteration 2 - UC 2 (18.-24. marts) Den oprindelig plan for Iteration 2 var at arbejde med UC3 (Book transportmiddel). Vi vurderede dog,

at det var uhensigtsmæssigt, at implementere bookinglogik på et fundament vi ikke var tilfredse med. Derfor blev de tilbageværende tasks videreført til iteration 2.

I perioden onsdag d. 18 marts til fredag d. 20 marts blev tiden brugt på refactoring af arkitekturen. Dette arbejde var ikke direkte task-drevet og bidrog til yderligere forskydning i tidsplanen.

Refaktoringen resulterede dog i et mere generisk og genanvendeligt design. Eksempelvis kan en fremtidig BookingRepository implementeres ved at nedarve fra en FileHandler uden ændringer af eksisterende kode.

Mandag d. 23 marts blev WPF-delen færdiggjort for tasken, og der blev implementeret en AppFacade, for at skjule vores servicelag bag en samlet grænseflade samt oprettelse af DI-root'en.

Forløbet synliggjorde udfordringerne ved at følge UP i en begrænset tidsramme, særligt når der samtidig er fokus på implementering af design patterns og arkitektur generelt. På baggrund af dette besluttede vi, i slutningen af iteration 2, at reducere afvige af den planlagte proces og for at sikre implementeringen af designpattern.

4 Arkitektur og design

4.1 Composite Pattern

I vores første iteration af projektet var udgangspunktet use casen UC2 - 'Se ledige transportmidler for et bestemt tidspunkt'. For at kunne estimere implementeringen mere præcist, foldede vi den ud, så den blev flere mindre tasks. Vi oprettede domæneklasser for Car, Bike og Employee og dertilhørende json-filer. For at sikre en så fleksibel og vedligeholdelsesvenlig arkitektur som muligt, valgte vi at implementere et repository pattern. Formålet var at abstrahere dataadgangen, så applikationslaget ikke har direkte afhængighed til datakilden. Repositories kan virke overflødigt i vores indtil nu simple system, men gør det skalerbart til en fremtidig database som datakilde uden at skulle ændre logik i vores servicelag. Samtidig bidrager det til at holde vores domæneklasser rene og fri for datalagringslogik. Vi introducerede nu interfacet, `IRepository<T>`, fordi vi havde flere repositories med ens funktionalitet - de læser json-filerne og deserialiserer dem til objekter. For at undgå gentagelse oprettede vi den abstrakte `FileHandler`-klasse, som de tre konkrete repositories nedarver fra. Til at starte med implementerede vi en superklasse, `BaseVehicleRepository`, som `CarRepository` og `BikeRepository` nedarvede fra. Den blev introduceret for at samle fælles funktionalitet for transportmidler, mens selve polymorfien blev håndteret gennem `IVehicleProvider`, hvor konkrete typer blev opcastet til `IVehicle`. Med introduktionen af interfacet, `IVehicleProvider`, blev ansvaret flyttet væk fra superklassen, da det var interfacet, som `VehicleService` afhang af. Vores base-klasse gjorde det dog stadig muligt at genbruge logik mellem repositories.

Under implementeringen blev vi med hjælp fra et genereret klassediagram opmærksomme på den unødvendige kompleksitet i vores design, hvilket gjorde både superklassen og interfacet overflødige. På dette tidspunkt bestod vores arkitektur stadig af to abstraktioner - `IRepository <T>` og `IVehicleProvider`, som servicelaget var afhængig af for at kunne håndtere forskellige transportmidler, da `IRepository<Car>` og `IRepository<Bike>` ikke kunne behandles som `IRepository<IVehicle>`. Klassediagrammet gjorde det tydeligt, at superklassen og `IVehicleProvider` kun eksisterede i systemet for at løse det problem. I et forsøg på at samle flere forskellige transportmidler under en struktur, implementerede vi klassen

`CompositeVehicleProvider`, der samlede flere `IVehicleProvider`-instanser og fungerede som et composite pattern.

```
  // IVehicleProvider - interfacet som samlingspunktet afhang af

  public interface IVehicleProvider
  {
      IEnumerable<IVehicle> LoadVehicles();
  }

  // BaseVehicleRepository - superklassen der nedarver til Car- og
BikeRepository

  public abstract class BaseVehicleRepository<T> : FileHandler<T>,
IVehicleProvider
  {
      public BaseVehicleRepository(string path) : base(path) { }
      public IEnumerable<IVehicle> LoadVehicles() ⇒
Load().Cast<IVehicle>();
  }

  // CompositeVehicleProvider - samler flere IVehicleProvider-instanser og
fungerer som et composite pattern

  public class CompositeVehicleProvider : IVehicleProvider
  {
      private readonly IEnumerable<IVehicleProvider> _providers;

      public CompositeVehicleProvider(IEnumerable<IVehicleProvider>
providers)
      {
          _providers = providers;
      }

      public IEnumerable<IVehicle> LoadVehicles()
      {
          var allVehicles = new List<IVehicle>();
          foreach (var provider in _providers)
          {
              allVehicles.AddRange(provider.LoadVehicles());
          }
          return allVehicles;
      }
  }
```

Efter indplementeringen af kovarians blev det muligt at generalisere vores composite pattern og skabe `CompositeRepository<T>` som en generisk løsning. Med kovarians `<out T>` i `IRepository<T>` blev det muligt at behandle repositories af specifikke typer, som Bike

og Car, som repositories af deres fælles supertype `IVehicle`. Med denne vigtige ændring blev designet åbent for nye samlinger af repositories i senere iterationer. Det betyder, at compileren nu tillader, at `IRepository<Car>` og `IRepository<Bike>`, samt alle fremtidige transportmidler, der implementerer IVehicle, kan behandles som `IRepository<IVehicle>`, og samtidig risikerer vi ikke, at vores employees bliver behandlet som transportmidler, fordi de ikke er en del af samme typehierarki.

Med indførelsen af kovarians kunne vi fjerne `IVehicleProvider` og `BaseVehicleRepository` og erstatte dem med en generisk `CompositeRepository<T>`. Refaktoringen til kovarians gjorde samtidig metoden `LoadVehicles()` overflødig, da behovet for runtime-casting til `IVehicle` blev erstattet af compile-time kontrol. Metoden var `BaseVehicleRepository`'s eneste ansvar - derfor kunne klassen fjernes.

```
// IRepository med kovarians (out T) - muliggør at IRepository<Car> kan
behandles som IRepository<IVehicle>

public interface IRepository<out T>
{
    IEnumerable<T> Load();
}

// CompositeRepository<T> - generisk composite pattern. T er covariant: Car
og Bike kan bruges som IVehicle, men Employee kan ikke, fordi den ikke
implementerer IVehicle

public class CompositeRepository<T> : IRepository<T>
{
    private readonly IEnumerable<IRepository<T>> _repositories;

    public CompositeRepository(IEnumerable<IRepository<T>> repositories)
    {
        _repositories = repositories;
    }

    public IEnumerable<T> Load()
    {
        return _repositories.SelectMany(r => r.Load());
    }
}

// VehicleService afhænger nu af IRepository<IVehicle> frem for
IVehicleProvider
public class VehicleService
```

```
{
    private readonly IRepository<IVehicle> _repository;

    public VehicleService(IRepository<IVehicle> repository)
    {
        _repository = repository;
    }

    public IEnumerable<IVehicle> GetVehicles()
    {
        return _repository.Load();
    }
}
```

Det ender med at give os et generisk Composite Pattern, hvor servicelaget kan hente alle objekter af en type uden at have kendskab til konkrete repository-implementationer.

4.2 Facade Pattern

5 Diskussion

5.1 Andre Patterns

5.2 SOLID og GRASP

6 Refleksion

6.1 Agil metode i et læringsscenarie

6.2 Task-formuleringer

7 Litteraturliste

Larman, C. (u.å.). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd ed). Prentice Hall PTR, c2005.

8 Bilag

Bilag A - SMS

Matias:

Det kommer jeg helt sikkert til. Tænker at vi kan snakke lidt om det på lørdag. Indtil da vil jeg gerne lige høre om det jeg hørte dig sige sidste gang er nogenlunde rigtigt: - Dig og dine kollegaer har daglige besøgslister på borgere i kommunen. - Til transport har i elcykler (nok til alle) og bil (til deling). - I skal selv beslutte rækkefølgen af jeres besøg. - I skal selv koordinere, kollegaer iblandt, hvem der bruger bil hvornår.

Lauras svar

Ja vi har alle 14 medarbejdere en borgerliste. Vi har 4 cykler, som vi deler med botilbuddet. Derudover har vi 4 biler til deling kun i Bostøtten. Nogle bruger også egen bil. Alle transportmidler bliver booket i Outlook. Der er nogle retningslinjer for hvor lang tid vi må booke frem mv. Vi koordinere selv hvornår borgerbesøg skal ligge. Men vi koordinere med hinanden på kryds og tværs. Da vi ofte deler en borger, så en har en borger i ulige uger og den anden i lige uger. Håber det giver mening